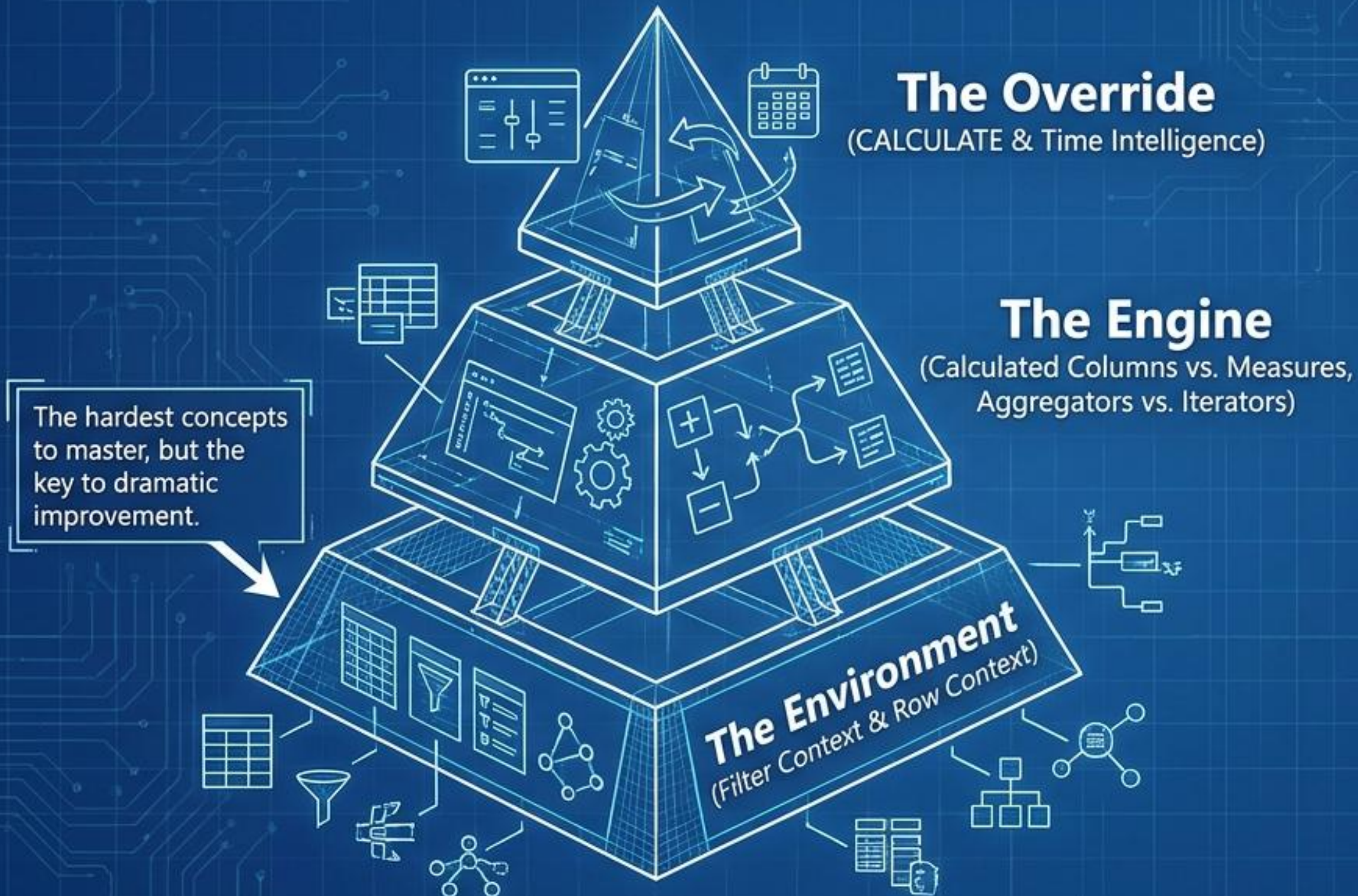


Power BI: Modeling with DAX

Mastering the Hidden Architecture of Context and Calculation

dr Zbigniew Galar

DAX is Simple, But Not Easy



Data Analysis Expressions uses syntax similar to Excel, but operates on entirely different physics.

The challenge isn't learning the functions.

It is visualizing the invisible context surrounding the data before the calculation even fires.

Anatomy of a Measure: The Syntax Blueprint

Measure Name

Rule: Measures are enclosed in square brackets when referenced, but never prefixed with a table name.

Function

Rule: Not case-sensitive. Tells DAX what operation to perform.

```
All_Sales = SUM('Orders Details'[LineTotal])
```

Table Name

Rule: Always fully qualify columns with table names. Single quotes are mandatory if the name contains a space.

Column Name

Rule: Always enclosed in square brackets.

The screenshot shows the Microsoft Power BI Desktop interface. The formula bar at the top displays the measure: `All_Sales = SUM('Orders Details'[LineTotal])`. Below the formula bar, a data table is visible with columns: OrderID, ProductID, UnitPrice, Quantity, Discount, Subtotal, DiscountAmount, and LineTotal. The table contains five rows of data. On the right side, the 'Fields' pane shows the 'Orders Details' table expanded, with 'Discount', 'DiscountValue', 'LineTotal', 'Measure', and 'OrderID' listed.

OrderID	ProductID	UnitPrice	Quantity	Discount	Subtotal	DiscountAmount	LineTotal
20999	20	\$18.00	20	0.0000	\$290	20.00	\$380.00
20521	23	218.00	3	0.0000	254	20.00	\$23.00
20230	35	218.00	20	0.0000	\$900	20.00	\$580.00
20346	23	218.00	20	0.0000	\$540	20.00	\$380.00
20546	26	\$18.00	20	0.0000	\$540	20.00	\$580.00

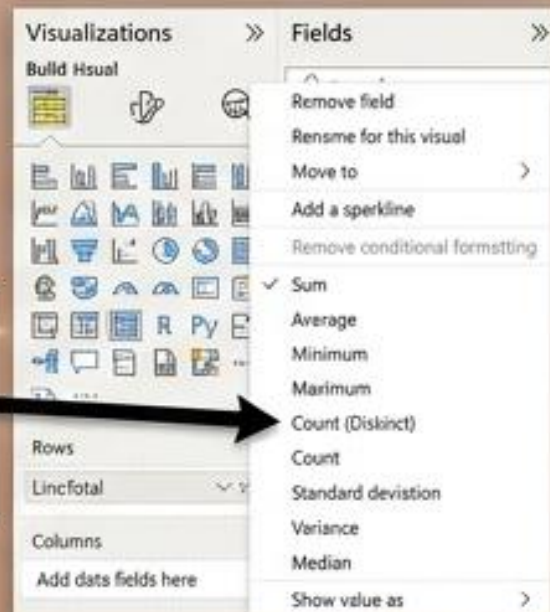
The Architecture of Trust: Implicit vs. Explicit Measures



Fragile: Implicit Measures

Created by dragging a raw numeric column directly into a visual.

Fatal Flaw: The default summarization behavior (Sum, Average, Count) is entirely exposed and can be accidentally altered by any report viewer.



When using an implicit measure, the aggregation behavior can be modified by the report viewer.

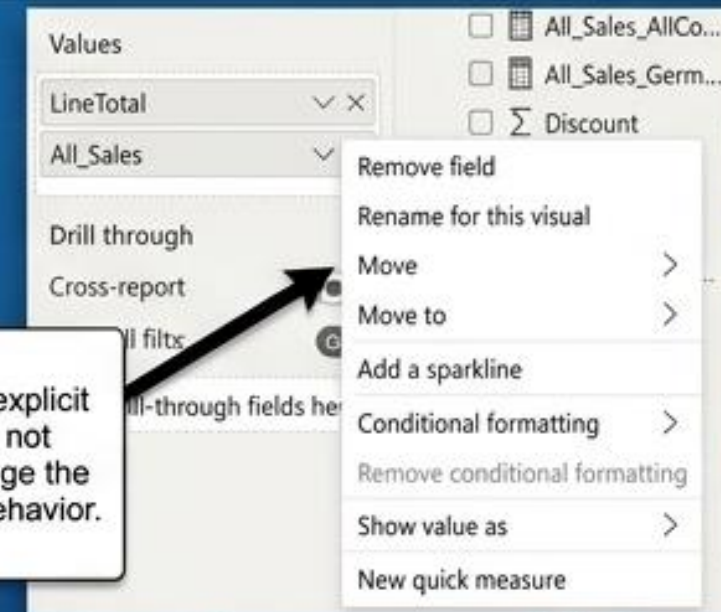


Robust: Explicit Measures

Created by writing a specific DAX formula

`All_Sales = SUM(...)`

Core Strength: The logic is locked down. The summarization behavior cannot be modified by the end-user.

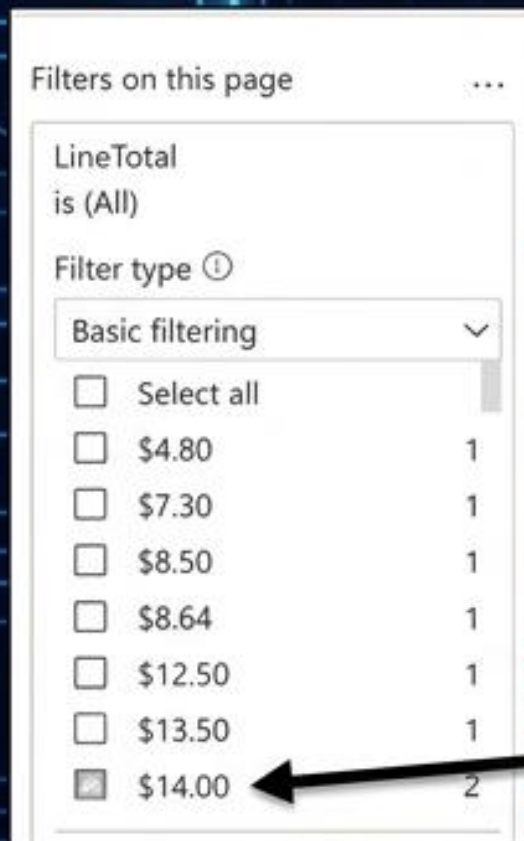


When using an explicit measure, it is not possible to change the summarization behavior.

Pro Tip: Once explicit measures are built, hide the base fields in the fact table. This forces report creators to use dimension tables and explicitly defined logic.

The Storage Matrix: Calculated Columns vs. Measures

	Calculated Columns	Measures
When it Calculates	During data refresh.	At query time (when a user clicks or loads a visual).
Storage Impact	High (Materialized and stored inside the data model, consumes memory).	Zero (Not materialized; requires no engine resources until explicitly used).
Mandatory Use Case	When you need to use the resulting values inside a Slicer or a Filter.	For aggregated totals displayed strictly inside visuals.



LineTotal (the calculated column) being used in the **Filter** pane.

Processing Engines: Bulk Aggregation vs. Row-by-Row Iteration

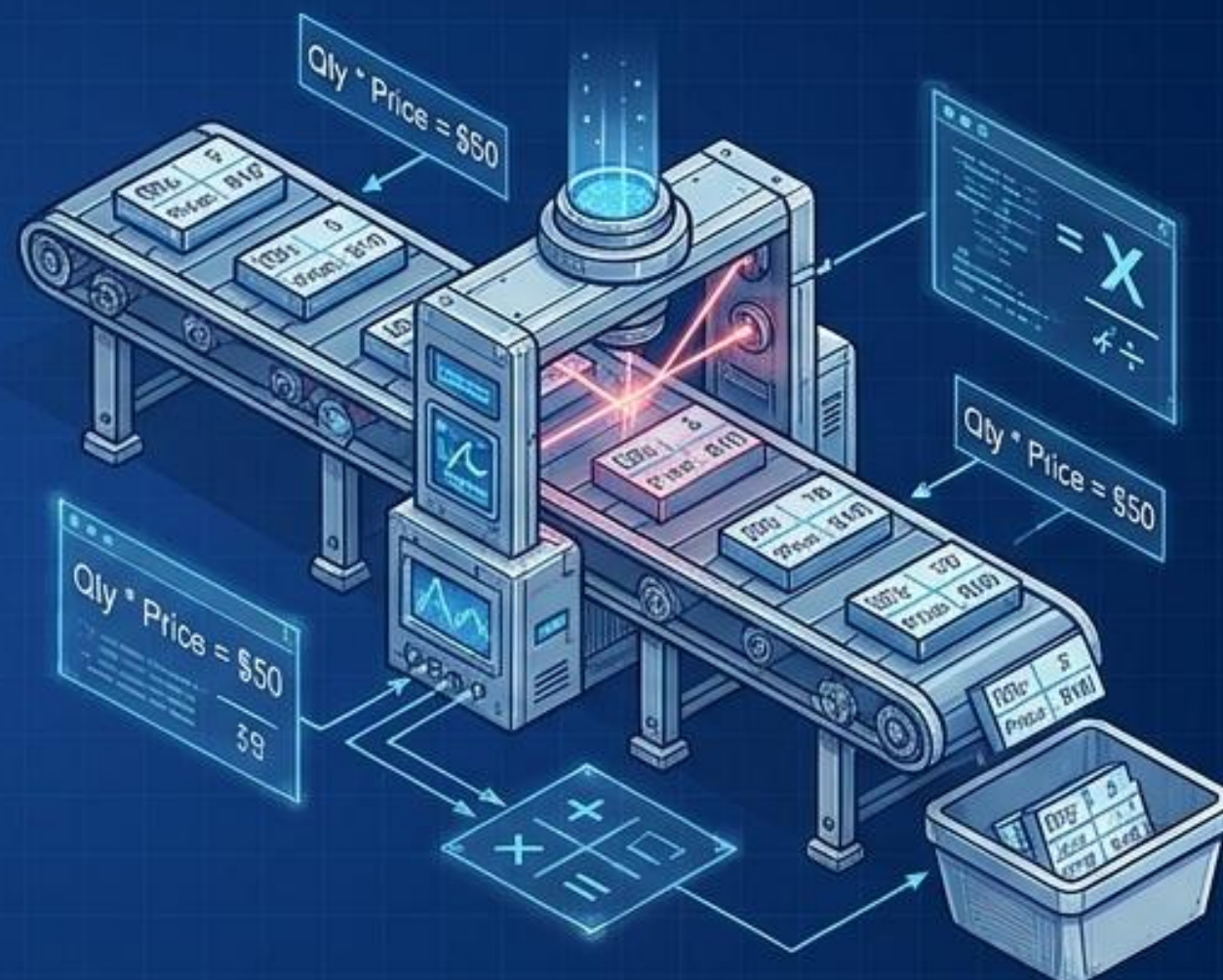
The Cookie Monster (Aggregators)



Standard functions gobble entire columns in one bite to produce a single result.

`SUM('Orders'[LineTotal])`

The Conveyor Belt (Iterators)

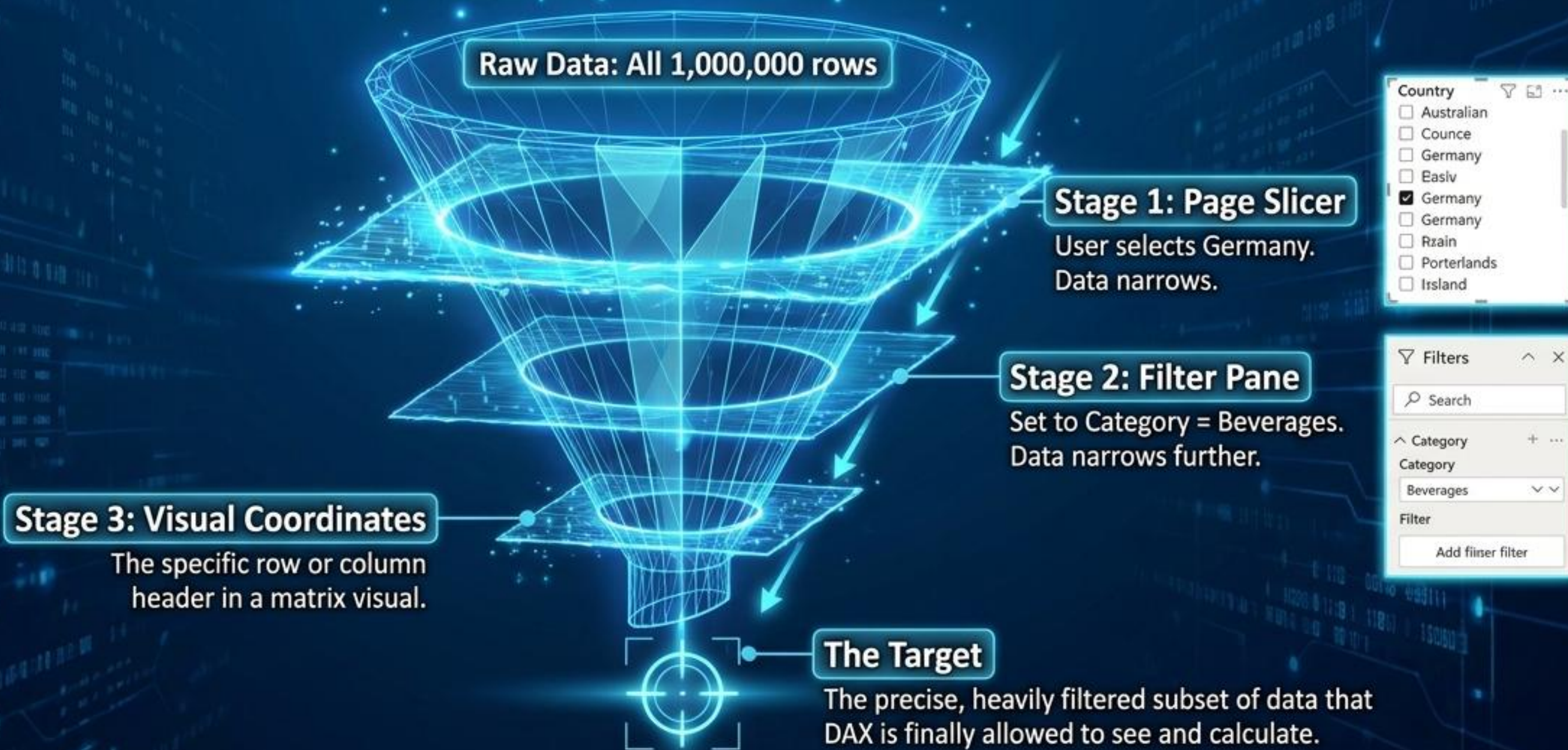


Iterator functions (ending in X) step through a table row-by-row to evaluate an expression before aggregating the final total.

`SUMX('Orders', 'Orders'[Qty] * 'Orders'[Price])`

CompanyName	All_Sales	LineTotal	LineTotalAsMeasure
Alfreds Futterkiste	\$4,273.00	\$4,273.00	\$4,273.00
Ana Trujillo Emparedados	\$1,402.95	\$1,402.95	\$1,402.95
Total	\$1,265,793.04	\$1,265,793.04	\$1,265,793.04

The Filter Context Funnel



Key Insight: Always be aware of the current filter context. DAX does not calculate the whole table; it only calculates the data surviving at the bottom of the funnel.

The Dual-Engine Matrix: Contexts Defined

Filter Context

What it does

Filters the data table down to a specific subset before math occurs.

Triggered by

Slicers, Visual interactions, Filter Pane, drill-throughs.

Mental Model

Which rows am I allowed to look at?

Row Context

What it does

Evaluates an expression for the current row.

Triggered by

Calculated Columns, Iterator functions (the 'X' functions).

Mental Model

Which specific row am I currently standing on?

**A measure cannot iterate row-by-row without a Row Context (requiring an X-function).
A measure cannot output a localized total without a Filter Context.**

Mastering CALCULATE: The Override Switch

`CALCULATE(Expression, [Filter1], ...)`

`CALCULATE(Expression, [Filter1], ...)`
Evaluates an expression in a context modified by filters.

The Concept

CALCULATE is the only tool that allows you to hijack and reprogram the Filter Context programmatically.

How it Works

It evaluates an expression, but first, it superimposes its own filter conditions onto the existing environment.



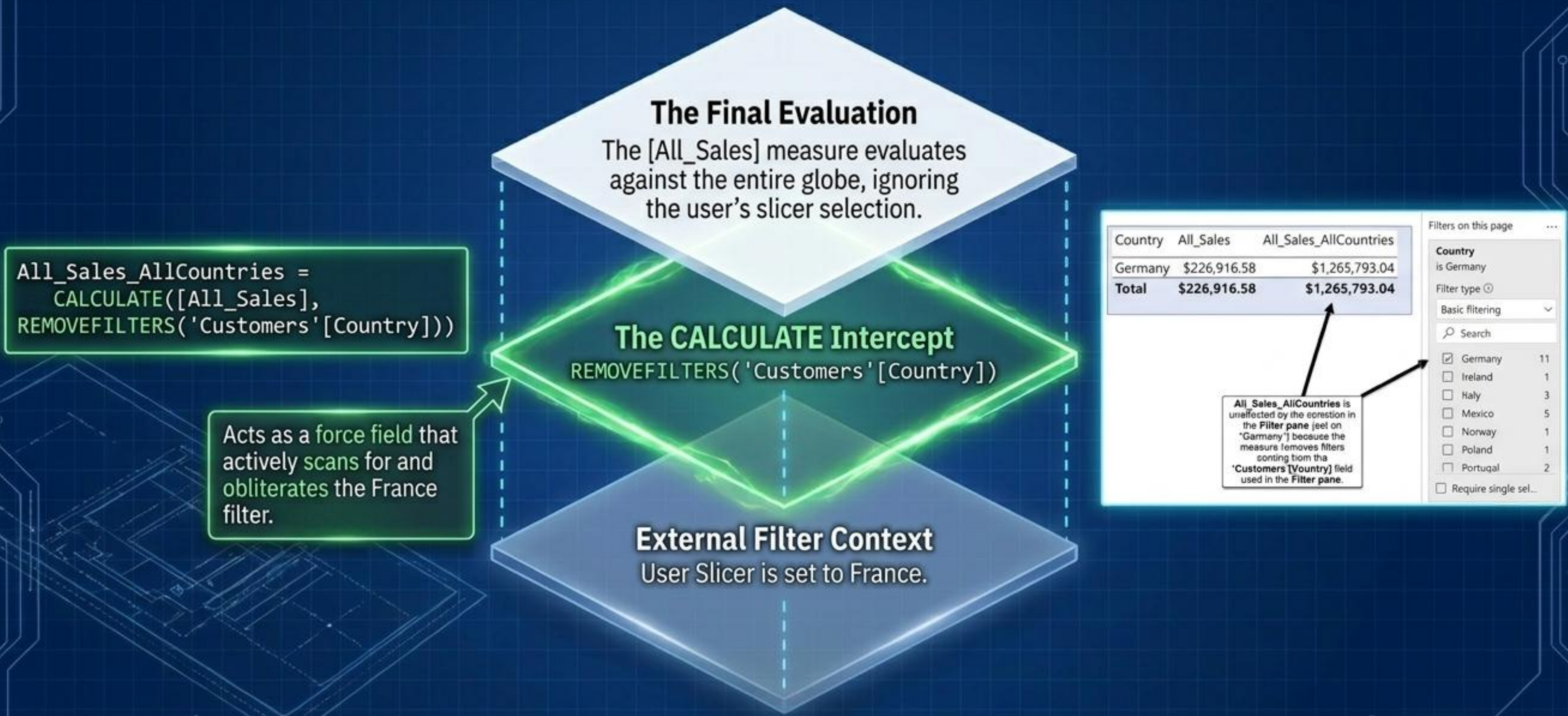
Argument 1:

The **Expression** (e.g., [All_Sales])

Optional Arguments:

Filter Modifiers (add/remove filters) or **Model Modifiers** (change relationships).

The Context Override Schematic





Modifying the Context: Erasing vs. Hardcoding

Strategy 1: Erasing

```
CALCULATE([All_Sales],  
  REMOVEFILTERS('Customers'[Country]))
```

Behavior

Ignores Country slicers, but respects other active slicers (e.g., if Category = Beverages, it still filters for Beverages).

Country	All_Sales	All_Sales_AllCountries
Germany	\$54,445.12	\$267,868.18
Total	\$54,445.12	\$267,868.18

CategoryName

☒ is Beverages 1

Country

☒ is Germany 11

All_Sales_AllCountries ignores the 'Germany' filter but respects the 'Beverages' filter.

Strategy 2: Hardcoding

```
CALCULATE([All_Sales],  
  'Customers'[Country] = "Germany")
```

Behavior

Forcibly overwrites any conflicting external filter context. If the user selects France, this measure aggressively overrides it to Germany.



The hardcoded measure displays Germany sales, matching the slicer.

The hardcoded measure refuses to change, continuing to show Germany sales even when France is selected.

Structural Alchemy: Temporary Model Modifiers

The Problem

A model can only have one active relationship between two tables. How do we report on Ship Date without breaking the entire model?

The Solution

USERELATIONSHIP. Used inside CALCULATE, it temporarily wakes up an inactive relationship just for the duration of that specific measure.



```
SampleMeasure = CALCULATE(  
    SUM('Orders'[Sales]),  
    USERELATIONSHIP('Date'[Date], 'Orders'[ShipDate])  
)
```


The Time Intelligence Requirement Map

Prerequisite Rule 1

Requires a continuous, marked Date Table (one row for every day between earliest and latest date).

Prerequisite Rule 2

A field from the Date Table must be included on the visual's axis or matrix.



Success

Correct Context Manipulation

Year	All Sales	YTD Sales	Sales Last Year
1996	\$208,083.97	\$208,083.97	
1997	\$617,085.20	\$617,085.20	\$208,083.97
1998	\$440,623.87	\$440,623.87	\$617,085.20
Total	\$1,265,793.04	\$440,623.87	\$825,169.17



Failure



The Opaque Warning

If you use TOTALYTD without a date field context, DAX returns an ambiguous, useless aggregate number.

All Sales

YTD Sales

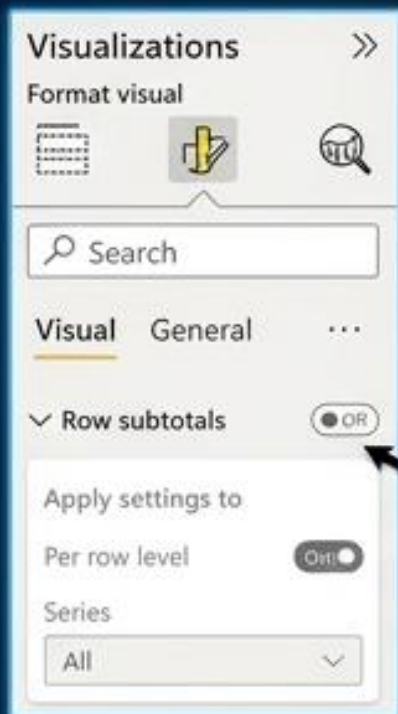
\$1,265,793.04	\$440,623.87
----------------	--------------

The Scope Hierarchy Tracker: Taming Grand Totals

The Problem

Some values shouldn't be added together at a grand total level (e.g., snapshot bank balances).

Turning off visual subtotals is clumsy and affects all columns.



It is possible to turn off the subtotals in a table visual, but this turns off the subtotals for every field used on the visual.



The Solution

ISINSCOPE wrapped in an IF statement.

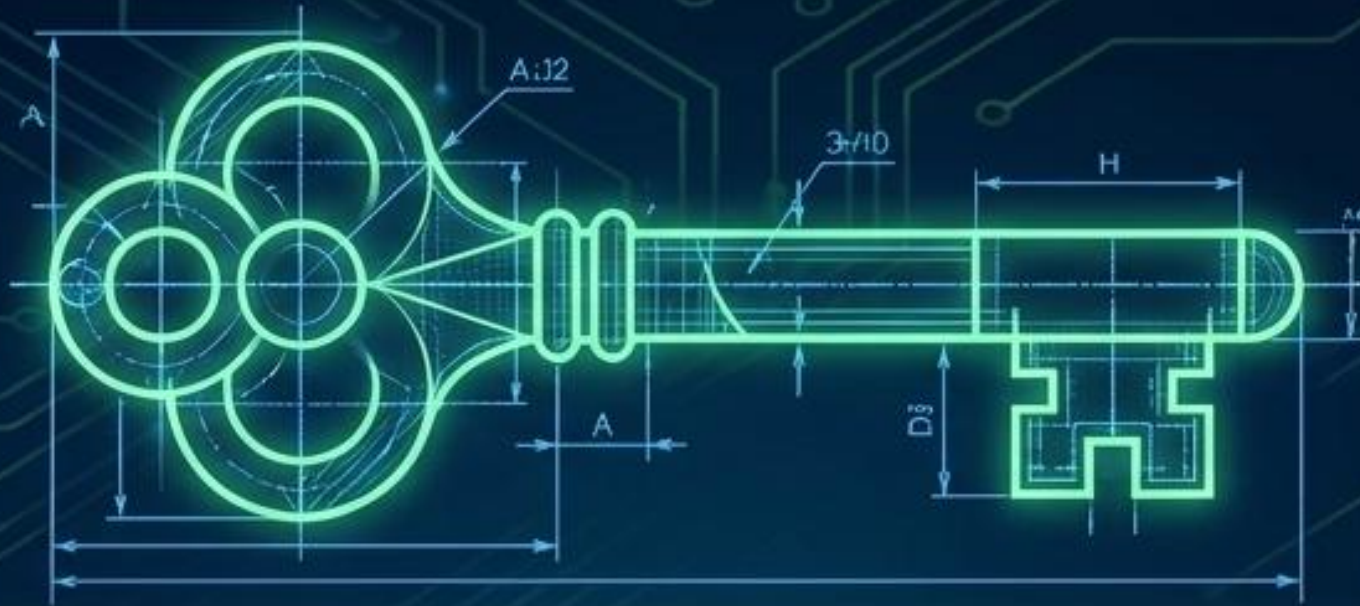
How it Works:

DAX checks: 'Is the current row grouped by City?'

If true, calculate the number.
If false (at the Grand Total line), return a blank.

Country	Address_All_Sales	City_All_Sales	Country_All_Sales
Australia			\$172,086.55
Melbourne		\$106,459.78	\$106,459.78
74 Rose St.	\$106,459.78	\$106,459.78	\$106,459.78
Moonie Ponds			
Sydney		\$65,626.77	\$65,626.77
170 Prince Edward	\$65,626.77	\$65,626.77	\$65,626.77
Hunter's Hill			
Brazil			\$4,504.37
São Paulo		\$4,504.37	\$4,504.37
Av. das Americanas	\$4,504.37	\$4,504.37	\$4,504.37
12.890			
Canada			\$83,742.21
Total			\$224,151.64

The Master Key to DAX



DAX provides hundreds of functions—from Statistical to Financial—but they all obey the same underlying physics.

Context Dictates Everything:

A formula's result is entirely dependent on the invisible filters surrounding it.

Explicit over Implicit:

Lock down logic with code; never rely on UI drag-and-drop.

Control Context with CALCULATE:

The defining skill of a DAX professional is mastering how CALCULATE overrides standard reality.

“Focus on understanding filter and row contexts. These are the hardest concepts to master, but once you do, your DAX will improve dramatically. – dr Zbigniew Galar”